

Roomify

AI-Powered Architectural Visualization Platform

Group Project Proposal

202512003
202512037
202512046
202512060
202512087
202512091
202512101
202512106

1. Description

Roomify is a web-based, AI-powered platform that converts 2D architectural floor plans into realistic, photorealistic 3D visualizations. It removes the time, cost, and specialist skill traditionally required for architectural rendering by letting users upload a flat floor plan and receive an AI-generated 3D render in minutes.

Alongside the core rendering engine, the platform provides permanent cloud media hosting, a personal project gallery, side-by-side input/output comparison, granular privacy controls, a public community feed for sharing work, and ownership mapping and export tools that support real-world design workflows.

2. Application Type

- Primary platform: Web application (React 19 + TypeScript + Vite), suited to the admin/creator-facing workflow of uploading plans, managing projects, and reviewing renders.
- Responsive design: The web UI is fully responsive (TailwindCSS) so it is also usable on tablets and mobile browsers for client-side viewing and sharing, without requiring a separate native mobile app in the initial scope.
- Deployment: Containerized with Docker for consistent deployment; backend services run as serverless Workers on the Puter cloud platform.

3. Motivation / Problem Statement

Producing a realistic 3D visualization from a 2D floor plan traditionally requires specialized CAD/rendering software, trained 3D artists, and hours-to-days of manual modeling and lighting work. This creates a bottleneck for:

- Architects and designers who need to quickly validate design concepts or produce client-ready visuals under tight deadlines.
- Architecture students who lack access to expensive rendering software or advanced 3D modeling skills.
- Clients and stakeholders who struggle to interpret flat, technical 2D drawings and want an intuitive way to "see" a space before it is built.

Roomify addresses this gap by using AI models (Claude and Gemini) to automate the 2D-to-3D transformation, making photorealistic visualization accessible, fast, and low-cost for non-specialists while still fitting into professional workflows through export and project-management features.

4. Target Users and Use Cases

Target Users

- Architects and architectural firms producing client presentations.
- Interior and exterior designers exploring layout options.
- Architecture and design students working on coursework or portfolios.
- Clients, investors, and other non-technical stakeholders reviewing proposed designs.
- Real-estate developers and agents visualizing properties from floor plans.

Use Cases

- Upload a scanned or digital 2D floor plan and generate a photorealistic 3D render for a client pitch.
- Compare multiple design iterations side-by-side to decide on layout or finish options.
- Maintain a private, organized gallery of past projects for a firm's internal reference library.
- Share a finished visualization publicly on the community feed to showcase work or gather feedback.
- Export a generated render for direct inclusion in a proposal, portfolio, or presentation deck.

5. Scope and High-Level Features

In Scope

- 2D-to-3D Visualization: AI-driven transformation of an uploaded floor plan into a photorealistic 3D render using Claude and Gemini models.
- Persistent Media Hosting: Permanent, publicly addressable URLs for every uploaded input and generated output, using Puter's storage layer.
- Dynamic Project Gallery: A personalized workspace listing each user's past visualizations with metadata (title, date, status).
- Side-by-Side Comparison: An interactive viewer that places the original 2D input next to the generated 3D output.
- Privacy Controls: Per-project public/private visibility toggles giving users control over what is shared.
- Global Community Feed: A public discovery feed where users can browse and share projects marked public.
- Ownership Mapping: Metadata linking every project and asset to its owning user ID for secure, scoped access.
- Export Functionality: One-click download of generated renders in formats suitable for presentations and external workflows.
- Authentication & account management, responsive UI, and basic usage analytics for the team's own monitoring.

Out of Scope (for this semester)

- Native iOS/Android applications (a fully responsive web app is delivered instead).
- Real-time multi-user collaborative editing of a single project.
- Full BIM/CAD interoperability (e.g., direct .dwg/.rvt import) beyond common image formats.
- Payment/billing and subscription management.

6. Technology Stack

Frontend

- React 19 with TypeScript for component-based, type-safe UI development.
- Vite as the build tool and development server.
- TailwindCSS for utility-first, responsive styling.

Backend / Cloud Infrastructure

- Puter platform: serverless Workers, permanent file/object storage, and a high-performance key-value (KV) database for project and user metadata.
- Puter.js SDK for frontend-to-cloud-service integration (auth, storage, KV, hosted AI models).

AI Models

- Anthropic Claude and Google Gemini, used for the 2D floor-plan analysis and 3D image generation pipeline.

Tooling & DevOps

- Docker for containerized builds and deployment.
- CodeRabbit for AI-assisted automated code review.
- Git/GitHub for version control and project/task tracking.

7. Roles and Responsibilities

Each of the 8 team members owns a primary module while contributing to shared code review, testing, and documentation, targeting roughly 200 person-hours per member across the semester.

Member	Role	Key Responsibilities
Member 1	Team Lead / Project Manager	Overall coordination, sprint planning, task assignment and tracking, client/instructor liaison, final integration sign-off, risk and timeline management.
Member 2	Frontend Lead (React/TypeScript)	Application shell, routing, global state and layout using React 19, Vite and TypeScript; component architecture and code review standards.
Member 3	UI/UX Developer	TailwindCSS design system, responsive layouts, upload/comparison and gallery screens, accessibility and visual consistency across the app.

Member	Role	Key Responsibilities
Member 4	AI Integration Engineer	Integration of Claude and Gemini models for 2D-to-3D transformation logic, prompt engineering, output validation and error handling.
Member 5	Backend / Cloud Infrastructure Engineer	Puter serverless Workers, permanent media hosting pipeline, KV database schema for project and user metadata, environment/config management.
Member 6	Authentication & Data Ownership Engineer	User authentication, ownership mapping, privacy (public/private) toggles, access-control rules and per-user data isolation.
Member 7	Community Feed & Export Features Developer	Global community feed, project sharing, gallery browsing/search, export/download functionality for renders.
Member 8	QA / DevOps & Documentation Lead	Test planning and execution, CodeRabbit-assisted code review, Docker build and deployment pipeline, README/user documentation, demo preparation.

8. Timeline with Milestones

The project is planned over a 15-week semester in six phases. Effort estimates are per-member averages and sum to the required team-wide benchmark.

Phase	Duration	Milestones & Deliverables	Effort
Phase 1 <i>Planning & Design</i>	Weeks 1-2	Requirements finalization, wireframes & UI design system, tech-stack setup (React/Vite/TS/Tailwind), Puter environment provisioning, task breakdown & GitHub project board.	~25 hrs
Phase 2 <i>Core Infrastructure</i>	Weeks 3-5	Authentication flow, KV storage schema, permanent media hosting pipeline, serverless Worker scaffolding, base app shell and routing.	~45 hrs
Phase 3 <i>AI Rendering Pipeline</i>	Weeks 6-8	2D floor-plan upload flow, Claude/Gemini integration for 2D-to-3D generation, output storage and metadata linking, error handling & retries.	~50 hrs
Phase 4 <i>Core Features</i>	Weeks 9-11	Project gallery, side-by-side comparison viewer, privacy (public/private) controls, ownership mapping, export/download functionality.	~40 hrs
Phase 5 <i>Community & Polish</i>	Weeks 12-13	Global community feed, search/browse, UI polish and responsiveness, performance tuning, accessibility pass.	~25 hrs
Phase 6 <i>Testing & Delivery</i>	Weeks 14-15	End-to-end testing, bug fixing, CodeRabbit review pass, Docker deployment, documentation, final report and demo video/presentation.	~15 hrs

Total: 15 weeks per member, summing to approximately 200 person-hours per member and ~1,600 person-hours across the 8-member team, matching the program benchmark.

GitHub repo - <https://github.com/Sakinakheraj/Roomify>